☁️ Technical Architecture Overview

📣 Release Notes Feature — Technical Architecture

⏰ Read This First — One-Time Build vs. ♾️ Ongoing Use

Everything below is a **one-time engineering build**. Once shipped, publishing a new release note is a **permanent, zero-code, self-service activity** — Admins/PMs never need engineering again for routine content.

🛠️ One-Time (this document covers this)	🔁 Ongoing (every release, forever)
Who	Engineering
What	DB, stored procedures, API, React components below
Deploy needed?	Yes, once

> 📎 Full boundary table + "when IS a developer needed again" → page `Jul 1, 2024` in `release-notes-architecture.drawio`.

📑 **Table of Contents**

- 1. Purpose
- 2. Tech Stack Context
- 3. High-Level Architecture
- 4. Frontend Architecture
- 5. Backend Architecture
- 6. Database Schema
- 7. Data Flow Summary
- 8. Security Notes
- 9. Extension Points
- 10. Known v1 Limitations

🎯 1. Purpose

A DB-driven "What's New" system. Content is authored entirely through an Admin Panel CRUD screen.

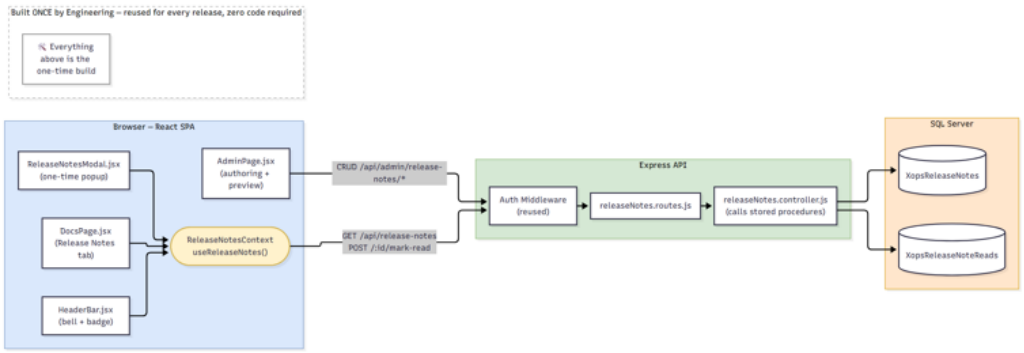
> 💡 **Key principle**: no code change or deploy is ever required to publish a new release note.

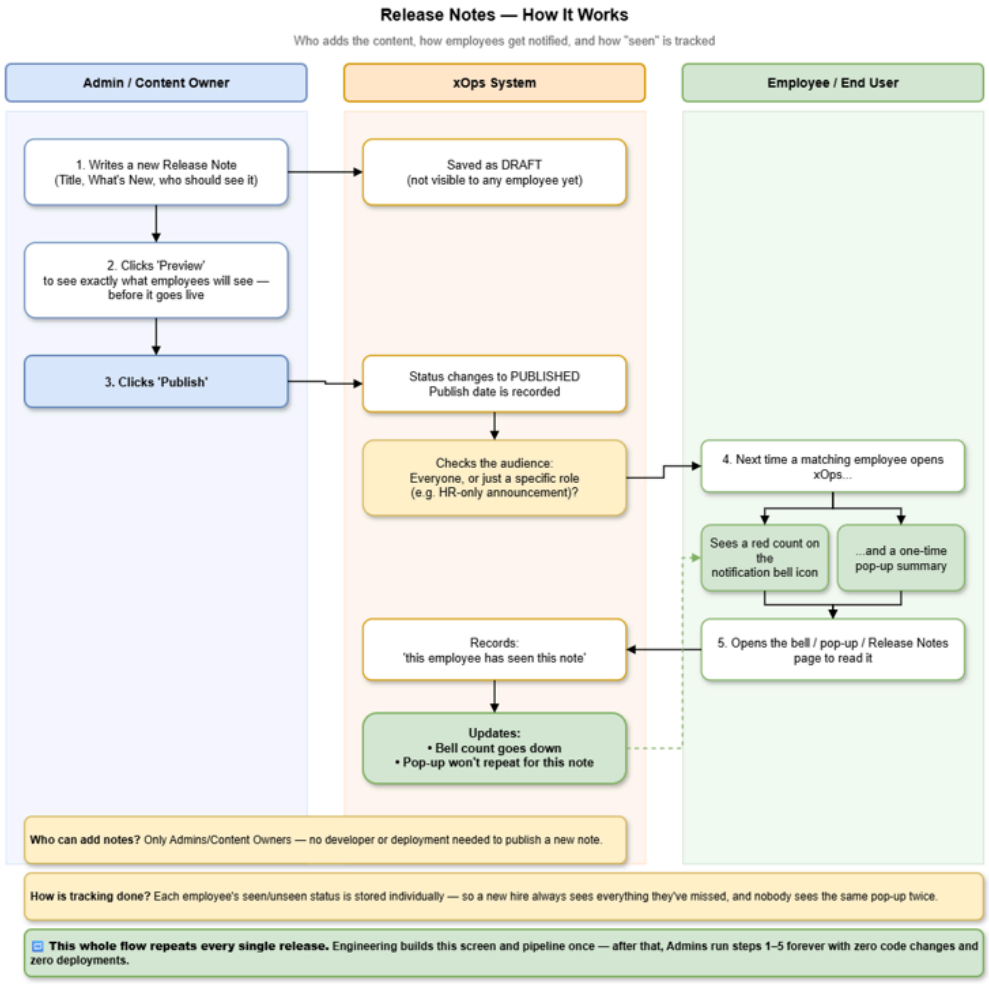
Surfaced in three places: a Docs page tab, a header bell icon with unread badge, and a one-time popup modal. Per-user read tracking powers all three.

⚙️ 2. Tech Stack Context

Layer	Technology
🎨 Frontend	React 18, MUI 5, React Router 6, axios, react-toastify
⚙️ Backend	Node.js, Express, `mssql` driver
🗄️ Database	SQL Server (MSSQL)
🔐 Auth	JWT via httpOnly cookie, existing auth middleware (reused as-is)

🏗️ 3. High-Level Architecture





📎 Full component/DB diagrams: `release-notes-architecture.drawio` (pages 1–6), `release-notes-db-schema.drawio` (ERD).

🖥️ 4. Frontend Architecture

📁 4.1 New/changed files

File	Status	Responsibility
`src/context/ReleaseNotesContext.jsx`	**new**	Single source of truth: fetches `GET /api/release-notes` once per session, derives `unreadCount`, exposes `markAsRead(id)`

`src/components/Layout/HeaderBar.jsx`	edit	Activates the existing (inert) `NotificationsIcon` — `Badge` + dropdown `Menu`, consumes the context
`src/Pages/DocsPage.jsx`	edit	Dynamic "Release Notes" section — fetches via context, renders via the existing `SECTION_RENDERERS` dispatcher
`src/docs/docsData.js`	edit	Retire the static commented-out `release-notes` stub once the dynamic tab is live
`src/Component/ReleaseNotesModal.jsx`	**new**	One-time-per-session popup, mirrors `HRReminderNotice.jsx`'s "fetch → show → dismiss → mark done" pattern
`src/Pages/AdminPage.jsx`	edit	New "Release Notes" tab: table + Add/Edit dialog + Edit/Preview toggle + block builder
`src/App.jsx`	edit	Mounts `ReleaseNotesProvider` (wrapping `<Layout>`) and `<ReleaseNotesModal />` alongside

		`<HRReminderNotice />`

⚙️ 4.2 `ReleaseNotesContext` — shape

```
// Returned by useReleaseNotes()
{
  notes,        // array of published, role-filtered notes (admin notes use a separate admin-only fetch)
  unreadNotes,  // notes.filter(n => !n.isRead)
  unreadCount,  // unreadNotes.length
  loading,
  error,
  markAsRead,   // (id) => optimistic local update + POST /api/release-notes/:id/mark-read
  refresh,      // manual refetch trigger
}
```

Why a Context and not 3 independent `useEffect` fetches: `HeaderBar`, `ReleaseNotesModal`, and `DocsPage` all mount on nearly every route. Independent fetches would mean 3x the network calls per page load. The Provider fetches once (on mount, post-auth) and all three subscribe to the same state — dismissing in the modal instantly updates the bell badge too.

🔁 4.3 Content rendering reuse

Release note bodies are stored as `ContentJson` — an array of blocks matching the **same shape** `docsData.js` already uses:

```
ContentJson = [
  { type: 'text',    heading, body },
  { type: 'list',    heading, items: [] },
  { type: 'callout', variant: 'info'|'warning'|'tip', body },
  { type: 'cards',   heading, cards: [{ icon, title, description }] },
]
```

Both `DocsPage.jsx` and `ReleaseNotesModal.jsx` render these through the **existing** `SECTION_RENDERERS` dispatcher (`DocsPage.jsx`) — no new rendering code needed. v1 admin block builder only exposes `text` / `list` / `callout` / `cards` (the other `docsData` types — `steps` / `table` / `flow` / `iconList` / `exportPanel` — are intentionally out of scope for v1 but the renderer already supports them if a future block type is added).

📝 4.4 Admin authoring UX

New "Release Notes" tab in `AdminPage.jsx`, following the existing HR Reminders tab pattern (table + `Dialog` + `handleSave*` / `handleDelete*` + `toast.success/warn/error`). The Add/Edit dialog has an **Edit / Preview** `Tabs` toggle — Preview mode renders the in-progress (unsaved) form state through the same `SECTION_RENDERERS` dispatcher, so the

author sees an exact live preview before hitting Save/Publish — pure client-side render, no round-trip.

📦 5. Backend Architecture

📂 5.1 New files

File	Responsibility
`routes/releaseNotes.routes.js`	Registers endpoints, mounted in `app.js` the same way other route modules are
`controllers/releaseNotes.controller.js`	Thin layer — calls stored procedures via `mssql`, no business logic/SQL inline
`scripts/migration-release-notes.sql`	Schema (tables + indexes), idempotent
`scripts/rollback-release-notes.sql`	Reverses the migration
`scripts/sp_ReleaseNotes_*.sql` (8 files)	One stored procedure per file, matching the existing `sp_UpsertAdminUser.sql` convention
`scripts/performance-check-release-notes.sql`	Seed data + index-usage verification queries

🛡️ 5.2 Why stored procedures (not inline SQL)

Every data operation goes through a stored procedure (`sp_ReleaseNotes_GetPublished` , `GetUnreadCount` , `MarkAsRead` , `GetAllForAdmin` , `Add` , `Update` , `TogglePublish` , `Delete`). The controller only does `pool.request().input(...).execute('sp_ReleaseNotes_X')` — no string-built or inline SQL in JS. This matches the codebase's existing admin-module convention (`sp_UpsertAdminUser` , `sp_AddDelegatedAccess`) and keeps query plans stable/reusable (parameter sniffing is predictable, and a DBA can tune/review procedures independently of app deploys).

🔗 5.3 Controller → stored procedure map

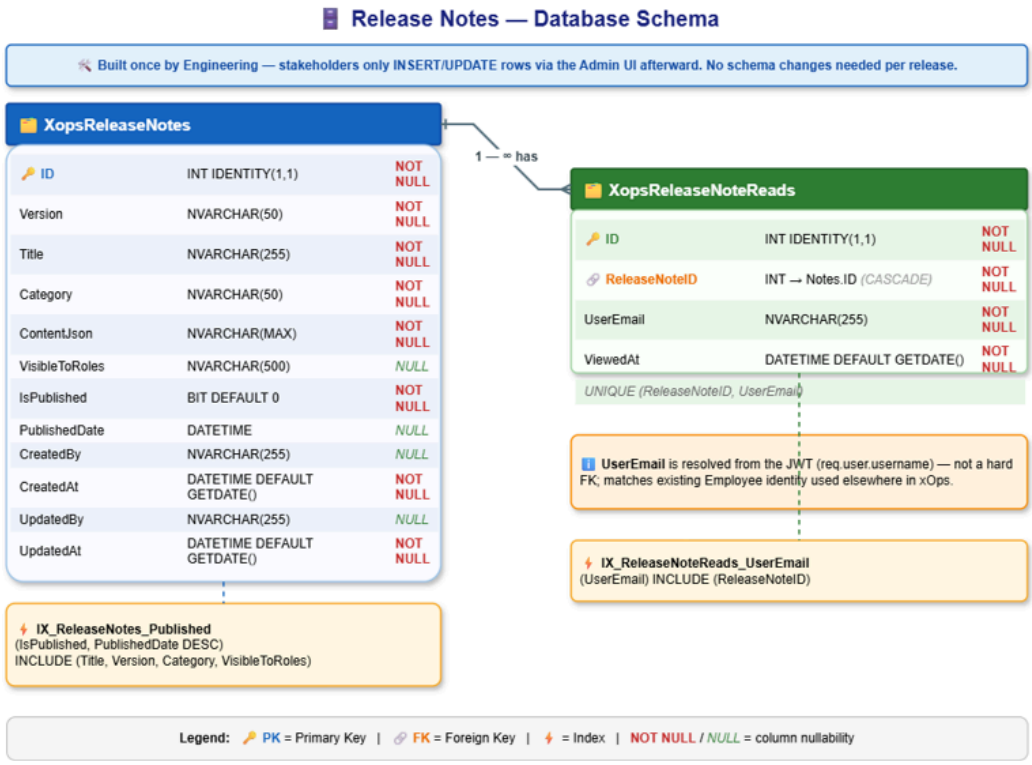
Controller function	Stored procedure	HTTP route
`getPublishedReleaseNotes`	`sp_ReleaseNotes_GetPublished`	`GET /api/release-notes`
`getUnreadCount`	`sp_ReleaseNotes_GetUnreadCount`	`GET /api/release-notes/unread-count`
`markAsRead`	`sp_ReleaseNotes_MarkAsRead`	`POST /api/release-notes/:id/mark-read`
`getAllForAdmin`	`sp_ReleaseNotes_GetAllForAdmin`	`GET /api/admin/release-notes`
`addReleaseNote`	`sp_ReleaseNotes_Add`	`POST /api/admin/release-notes/add`
`updateReleaseNote`	`sp_ReleaseNotes_Update`	`PUT /api/admin/release-notes/:id`
`togglePublish`	`sp_ReleaseNotes_TogglePublish`	`PATCH /api/admin/release-notes/:id/toggle-publish`
`deleteReleaseNote`	`sp_ReleaseNotes_Delete`	`DELETE /api/admin/release-notes/:id`

Role/audience filtering (`VisibleToRoles`) happens **inside the stored procedure** (`sp_ReleaseNotes_GetPublished` / `GetUnreadCount`), not in JS and not just hidden client-side — the API itself never returns role-restricted content to the wrong audience.

🔄 5.4 `/api/release-notes/*` vs `/api/admin/release-notes/*`

Route	Access
`/api/release-notes/*`	any authenticated user; auth middleware only.
`/api/admin/release-notes/*`	additionally requires the admin-role check already used by `admin.routes.js`.

🗄️ 6. Database Schema



> 🔑 **Indexes**

- `IX_ReleaseNotes_Published (IsPublished, PublishedDate DESC) INCLUDE (Title, Version, Category, VisibleToRoles)` — serves the main list query as a pure index seek.
- `IX_ReleaseNoteReads_UserEmail (UserEmail) INCLUDE (ReleaseNoteID)` — serves the unread-count lookup (the UNIQUE constraint's leading column is `ReleaseNoteID`, so it can't serve this direction).

> ⚠️ **Note**: `UserEmail` is **not** a hard FK to an employee table — it's resolved from the JWT (`req.user.username`) at request time, matching how user identity is referenced elsewhere in xOps.

🔄 7. Data Flow Summary

```mermaid
flowchart LR
    A["✍️ Admin Input"] --> B["Client Validation"] --> C["Serialize → ContentJson"] --> D["POST/PUT /api/admin/release-notes"] --> E["sp_ReleaseNotes_Add/Update"]
```

Write path (Admin): form input → client validation → serialize blocks to `ContentJson` → `POST/PUT /api/admin/release-notes` → *controller validates + resolves role string →* `sp_ReleaseNotes_Add` / `Update` → *row in* `XopsReleaseNotes`.

```mermaid
flowchart LR
    F["ReleaseNotesProvider mounts"] --> G["GET /api/release-notes"] --> H["sp_ReleaseNotes_GetPublished (role-filtered + isRead join)"] --> I["SECTION_RENDERERS (Docs / Bell / Popup)"]
```

Read path (User): `ReleaseNotesProvider` *mounts →* `GET /api/release-notes` → `sp_ReleaseNotes_GetPublished` *(role-filtered, LEFT JOIN* `XopsReleaseNoteReads` *for* `isRead`*) → rendered via* `SECTION_RENDERERS` *in Docs page / bell dropdown / popup.*

```mermaid
flowchart LR
    J["User opens/dismisses note"] --> K["markAsRead(id) (optimistic update)"] --> L["POST /:id/mark-read"] --> M["sp_ReleaseNotes_MarkAsRead (idempotent upsert)"]
```

Tracking path: reflected in the next `GetPublished` / `GetUnreadCount` *call.*

🔒 8. Security Notes (dev-relevant)

- All SQL access is parameterized via stored procedure `@params` — never string-concatenated.

- `ContentJson` is rendered via the fixed `SECTION_RENDERERS` component set — never `dangerouslySetInnerHTML` of raw admin input, so no stored-XSS surface even though content is admin-authored.
- Auth/authorization is **fully reused** from the existing middleware — no new auth mechanism was introduced.

✨ 9. Extension Points (for future developers)

- **New block type** (e.g. `table`): add a case to the admin block-builder form; the renderer (`SECTION_RENDERERS`) likely already supports it from the `docsData` system — check `DocsPage.jsx` first.
- **New audience granularity** (e.g. per-department instead of per-role): `VisibleToRoles` is a simple comma-separated string matched via `CHARINDEX` in the stored procedures — swapping to a join table would require a new migration + updating `sp_ReleaseNotes_GetPublished` / `GetUnreadCount` only (controller/frontend untouched).
- **Real-time push** (no polling/refresh needed): not implemented in v1 — `refresh()` on the context is manual/interval-based today; a websocket/SignalR push would plug into `ReleaseNotesContext` without touching the DB layer.

⚠️ 10. Known v1 Limitations

- No pagination on `sp_ReleaseNotes_GetAllForAdmin` — fine at current volume (admin-authored, low write rate); add `TOP N`/paging if the list grows large.
- Popup "show once" guard is session-based (client) backed by server-side read state — not a hard real-time cross-device sync (acceptable per product decision, see PM flow diagram).

Maintained alongside release-notes-architecture.drawio · xOps Engineering